# Classificação de complexidade textual em português com Machine Learning: modelo, interface web e algoritmo de mutação de texto

**Pedro Henrique Cezário de Melo**

Juiz de Fora, 16 de outubro de 2026

# Sumário

Resumo ..... 5
1. Introdução ..... 6
1.1 Contexto e problema ..... 6
1.2 Objetivos ..... 6
1.3 Componentes do projeto ..... 7
1.4 Visão geral da solução ..... 7
1.5 Métricas e referências de desempenho ..... 7
2. Dados ..... 8
2.1 Origem e organização ..... 8
2.2 Ingestão e tratamento ..... 8
2.3 Distribuição por classe ..... 8
2.4 Divisão em treino, validação e teste ..... 9
2.5 Comprimento dos textos ..... 10
2.6 Observações sobre a natureza dos textos ..... 11
3. Metodologia ..... 12
3.1 Visão geral das 30 features ..... 12
3.2 Processamento comum ..... 12
3.3 Pontuação ..... 13
3.4 Tamanho das sentenças ..... 13
3.5 Classes gramaticais ..... 14
3.6 Features lexicais ..... 14
3.7 Canonicidade (feature obrigatória) ..... 14
3.8 Protocolo de avaliação ..... 15
3.9 Algoritmos comparados ..... 16
4. Experimentos e resultados ..... 17
4.1 Resumo cronológico ..... 17
4.2 Desenvolvimento e correção da canonicidade ..... 17
4.3 Análise descritiva das 26 features iniciais ..... 18
4.4 Baseline: Random Forest com 26 features e balanceamento de classes ..... 20
4.5 Features lexicais: de 26 para 30 features ..... 22
4.6 Validação cruzada: o ganho é real, mas menor que o aparente ..... 24
4.7 Features de riqueza lexical: de 30 para 34 (resultado negativo) ..... 25

4.8 Comparação dos quatro algoritmos (CV, 30 features) ..... 27
4.9 Escolha do modelo final ..... 28
4.10 Registro consolidado de problemas e correções ..... 28
5. Resultado final (teste) ..... 29
5.1 Procedimento ..... 29
5.2 Resultados ..... 29
5.3 Matriz de confusão ..... 29
5.4 Interpretação ..... 30
6. Aplicação: interface web e algoritmo de mutação ..... 31
6.1 Interface (Streamlit) ..... 31
6.2 Algoritmo de mutação de texto ..... 32
6.3 Efeito esperado sobre as features e plano de avaliação ..... 34
6.4 Observações técnicas sobre a interface e o algoritmo ..... 35
7. Discussão e limitações ..... 37
7.1 Principais achados ..... 37
7.2 Canonicidade ..... 37
7.3 Confusão entre Médio e Superior ..... 37
7.4 Risco de aprendizado por atalho ..... 37
7.5 Limitações de dados e de avaliação ..... 38
8. Conclusão e trabalhos futuros ..... 39
8.1 Conclusão ..... 39
8.2 Trabalhos futuros ..... 39
9. Referência bibliográfica ..... 41
10. Anexos ..... 42
10.1 Estrutura do projeto e execução ..... 42
10.2 Ambiente ..... 42
10.3 Configuração do modelo final ..... 43
10.4 As 30 features, em ordem ..... 43
10.5 Tecnologias utilizadas ..... 43

## Resumo

Este relatório documenta o desenvolvimento de um sistema de classificação de textos em português por nível de complexidade (Ensino Fundamental I, Ensino Fundamental II, Ensino Médio e Ensino Superior). O sistema tem três componentes: (i) um modelo de Machine Learning supervisionado; (ii) uma interface web em Streamlit; e (iii) um algoritmo de mutação de texto que permite reclassificar uma versão modificada do texto de entrada.

O corpus final tem 2.060 textos (após a remoção de 16 duplicatas), divididos de forma estratificada em 1.442 textos de treino, 309 de validação e 309 de teste. Cada texto é representado por um vetor de 30 features linguísticas: pontuação, tamanho de sentenças, classes gramaticais, métricas lexicais e a feature obrigatória de canonicidade silábica. Quatro algoritmos foram comparados por validação cruzada estratificada de 5 dobras: SVC (F1 macro 0,785 ± 0,015), Random Forest (0,771 ± 0,016), Regressão Logística (0,742 ± 0,019) e MultinomialNB (0,342 ± 0,017). O SVC foi escolhido e, avaliado uma única vez no conjunto de teste, obteve **F1 macro de 0,778 e acurácia de 0,790**.

Os erros concentram-se entre classes adjacentes, sobretudo Médio ↔ Superior. A canonicidade, mantida por exigência do projeto, apresentou baixo poder discriminativo. O algoritmo de mutação foi implementado e integrado à interface, mas seu efeito sobre a classificação **não foi quantificado experimentalmente**; a Seção 6 descreve um protocolo de avaliação para essa etapa.

# 1. Introdução

## 1.1 Contexto e problema

Estimar o nível de complexidade de um texto tem aplicações em educação (adequação de materiais ao nível do estudante), curadoria de conteúdo e análise de legibilidade. Neste projeto, o problema é formulado como **classificação multiclasse supervisionada** com quatro categorias: Ensino Fundamental I, Ensino Fundamental II, Ensino Médio e Ensino Superior. As categorias possuem uma ordem natural, mas são tratadas como nominais: o modelo não recebe nenhuma informação sobre essa ordem.

Algoritmos clássicos de ML não consomem texto bruto; exigem uma representação numérica. A abordagem adotada é a **engenharia de features linguísticas explícitas**: cada texto é convertido em um vetor de 30 medidas interpretáveis. Essa escolha privilegia a interpretabilidade e a possibilidade de justificar cada variável, em detrimento de representações latentes (TF-IDF, embeddings), que ficam como trabalho futuro.

## 1.2 Objetivos

**Objetivo geral:** desenvolver um modelo de ML que classifique textos em português segundo a complexidade, executado em uma interface web na qual o usuário pode enviar um texto e, opcionalmente, aplicar um algoritmo de mutação para observar se a classificação se altera.

**Objetivos específicos:**

- implementar uma ingestão reprodutível dos dados locais e a divisão estratificada em treino, validação e teste;
- implementar a feature obrigatória de **canonicidade** e demais features linguísticas, validando-as com análise descritiva e importâncias;
- comparar quatro algoritmos (Random Forest, SVC, Regressão Logística e MultinomialNB) com a mesma validação cruzada;
- avaliar o modelo escolhido **uma única vez** em dados de teste;
- disponibilizar uma interface web (texto digitado ou arquivo `.txt`) que aciona o modelo;
- implementar o algoritmo de mutação de texto e permitir a reclassificação do texto modificado.

### 1.3 Componentes do projeto

| Arquivo | Papel |
|---|---|
| construindo_ML.ipynb | Ingestão, divisão, features, experimentos, validação cruzada, avaliação final e salvamento do modelo |
| Features.py | Extração das 30 features para um texto novo (usado pela interface; reproduz o código do notebook) |
| modelo_svc_complexidade.joblib | Pipeline treinado (StandardScaler + SVC) |
| mudança_texto.py | Algoritmo de mutação de texto: troca de gênero com concordância e substituição por sinônimos/antônimos |
| app.py | Interface Streamlit: entrada, mutação, classificação e exibição do resultado |

### 1.4 Visão geral da solução

Texto —► extrair_features_nlp —► vetor de 30 features —► StandardScaler —► SVC —► nível previsto

▲

└— (opcional) mutação de texto (mudança_texto.py), acionada por botão na interface

O treinamento ocorre offline, no notebook. A inferência ocorre na interface, que reutiliza **o mesmo código de extração de features** (Features.py) e o pipeline salvo em .joblib. Reaproveitar o código de extração evita a divergência entre treino e uso (train/serve skew): se o vetor fosse calculado de forma diferente na interface, o modelo receberia entradas fora da distribuição de treino.

### 1.5 Métricas e referências de desempenho

- **Acurácia:** fração de textos classificados corretamente.
- **Precision, recall e F1** por classe: precision é a fração de acertos entre os textos que o modelo atribuiu à classe; recall é a fração dos textos da classe que o modelo encontrou; F1 é a média harmônica dos dois.
- **F1 macro:** média aritmética do F1 das quatro classes, com o mesmo peso para cada uma. É a **métrica de decisão** do projeto, porque o corpus é desbalanceado (Seção 2.3) e a acurácia favorece as classes grandes.
- **Referências de piso:** um chute uniforme entre quatro classes acerta cerca de 25%; prever sempre a classe majoritária (Superior) acerta cerca de 39,5%.

# 2. Dados

### 2.1 Origem e organização

Os dados estão armazenados localmente em uma pasta principal (dataset_caed) com uma subpasta por categoria (1_Ensino_Fundamental_I, 2_Ensino_Fundamental_II, 3_Ensino_Medio, 4_Ensino_Superior). Cada subpasta contém arquivos .txt, um texto por arquivo, e o rótulo de cada texto é o nome da subpasta.

### 2.2 Ingestão e tratamento

A função carregar_dados percorre as quatro subpastas e monta um DataFrame com as colunas texto e complexidade. Os tratamentos aplicados foram:

| Problema | Tratamento | Resultado observado |
|---|---|---|
| Arquivos fora de UTF-8 | try/except UnicodeDecodeError: o arquivo é ignorado e um aviso agregado é impresso | Nenhum aviso foi emitido: nenhum arquivo foi descartado por encoding |
| Arquivos vazios | Descartados após strip() | — |
| Textos duplicados | drop_duplicates(subset='texto'), mantendo a primeira ocorrência | **16 duplicatas removidas** (2.076 → 2.060 textos não vazios) |
| Nenhum dado carregado | ValueError explícito | — |

A remoção de duplicatas evita **vazamento de dados** (*data leakage*): o mesmo texto no treino e no teste inflaria as métricas. Há duas ressalvas. Primeiro, a deduplicação considera só o texto; se um mesmo texto aparecesse em duas categorias, permaneceria apenas a primeira ocorrência, e esse caso não foi verificado. Segundo, uma pasta de categoria ausente é ignorada silenciosamente pelo teste os.path.exists, o que, em outro ambiente, poderia gerar um corpus incompleto sem aviso.

### 2.3 Distribuição por classe

| Classe | Total | % do corpus | Treino | Validação | Teste |
|---|---|---|---|---|---|
| Fundamental I | 294 | 14,3% | 206 | 44 | 44 |
| Fundamental II | 324 | 15,7% | 226 | 49 | 49 |
| Médio | 628 | 30,5% | 440 | 94 | 94 |
| Superior | 814 | 39,5% | 570 | 122 | 122 |
| **Total** | **2.060** | **100%** | **1.442** | **309** | **309** |

*Os totais por classe foram obtidos somando as três partições (treino,* support *da validação e* support *do teste).*

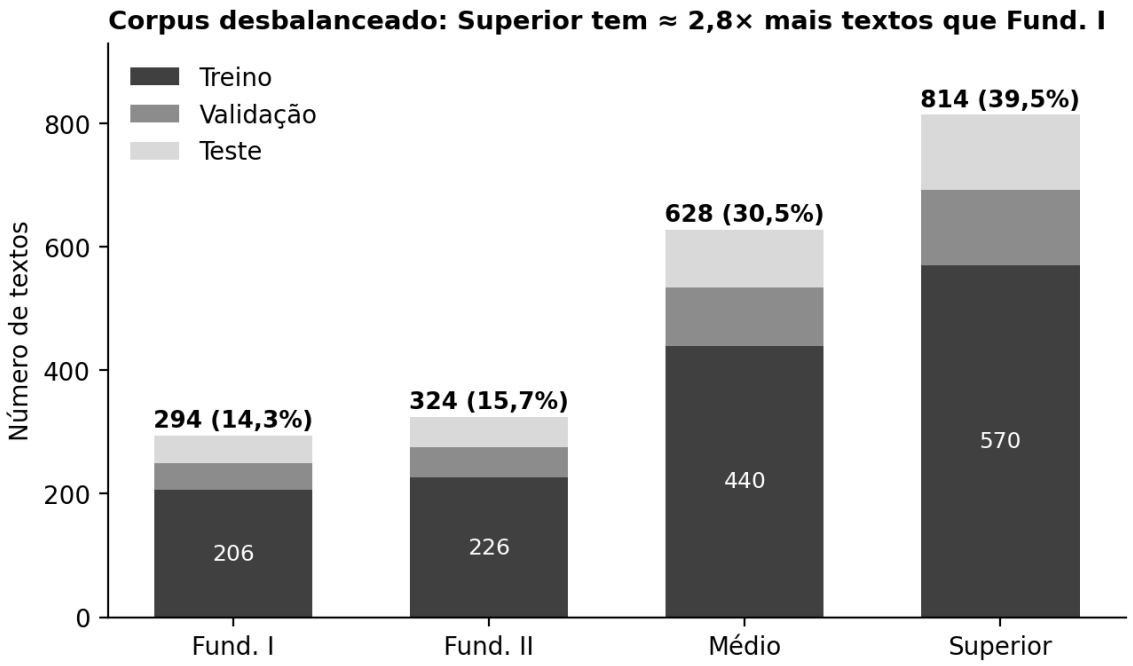



***Figura 1** — Distribuição dos 2.060 textos por classe e partição (treino, validação e teste). Fonte: contagens da Seção 2.3.*

O corpus é **desbalanceado**: Superior tem cerca de 2,8 vezes mais textos que Fundamental I. Consequências para o projeto: (i) a acurácia isolada é uma métrica enviesada, razão pela qual o F1 macro foi adotado; (ii) o parâmetro class_weight='balanced' foi testado e adotado nos modelos que o suportam (Seção 4.3).

### 2.4 Divisão em treino, validação e teste

A divisão foi feita em duas etapas, ambas **estratificadas** (preservando a proporção das quatro classes) e com random_state=42 (divisão reprodutível):

1. 15% do corpus foi separado como **teste**;
2. dos 85% restantes, test_size=0,1764 (≈ 0,15 / 0,85) foi separado como **validação**, resultando em ≈ 15% do corpus.

O resultado foi 1.442 textos de treino (70,0%), 309 de validação (15,0%) e 309 de teste (15,0%). Os papéis de cada partição foram:

- **treino:** ajuste dos modelos e estatísticas descritivas das features;
- **validação:** decisões de engenharia de features e de balanceamento;
- **teste:** mantido intocado até a avaliação final, usado uma única vez (Seção 5).

Na validação cruzada e no treinamento do modelo final, treino e validação foram combinados (1.751 textos); o teste permaneceu fora.

## 2.5 Comprimento dos textos

Tamanho em palavras (contagem por espaços) no conjunto de treino:

| Classe | Textos | Média | Desvio | Mín. | Mediana | Máx. |
|---|---|---|---|---|---|---|
| Fundamental I | 206 | 339,6 | 43,2 | 304 | 325,5 | 531 |
| Fundamental II | 226 | 395,4 | 86,5 | 298 | 377,5 | 608 |
| Médio | 440 | 382,8 | 80,5 | 300 | 349 | 660 |
| Superior | 570 | 412,1 | 97,3 | 303 | 369 | 789 |

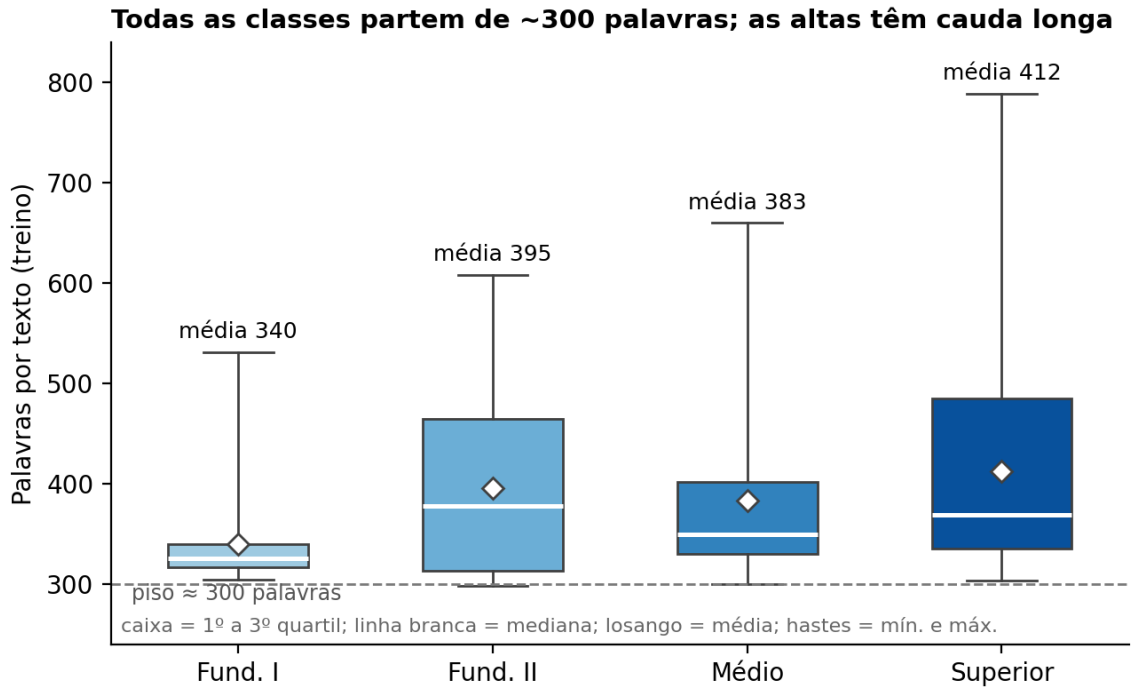



***Figura 2** — Comprimento dos textos (palavras separadas por espaços) no treino, por classe. O gráfico foi reconstruído a partir do mínimo, dos quartis, da mediana, do máximo e da média da tabela acima; as hastes vão do mínimo ao máximo. Fonte: Seção 2.5.*

Observações:

- todas as classes têm um **piso comum de aproximadamente 300 palavras**, o que sugere um corte aplicado na construção do corpus e reduz o ruído das proporções;
- a média cresce de 340 (Fund. I) para 412 (Superior), cerca de 21%, em grande parte por causa de caudas longas nas classes mais altas; as medianas ficam mais próximas (326 a 378);
- Fundamental I é a classe mais homogênea (desvio de 43 palavras).

Essa análise motivou quatro decisões: (a) todas as features são **proporções ou médias**, não contagens, para não codificar o tamanho do texto; (b) o TTR (*type-token ratio*) é calculado em **janela fixa de 300 palavras**; (c) o comprimento do texto **não** foi usado como feature, por ser um atalho do corpus e não uma medida linguística, com risco de não generalizar; (d) a interface exibe um aviso para textos com menos de 300 palavras.

### 2.6 Observações sobre a natureza dos textos

A inspeção das amostras impressas durante o desenvolvimento sugere origens editoriais heterogêneas entre os níveis. **Trata-se de observação qualitativa, não quantificada.**

- Nas amostras de Médio e Superior, vários textos começam com **cabeçalhos hierárquicos separados por barra** (por exemplo, Logística/Gestão de armazéns/... e Cálculo (Volume 2)/Formas paramétricas), típicos de material didático em formato wiki. Há também marcadores formula_N no lugar de expressões matemáticas.
- Nas amostras de Fundamental II, alguns textos começam com um **rótulo de área seguido de ponto** (Biologia., Sociologia.).
- Nas amostras de Fundamental I e II predominam textos biográficos, jornalísticos, narrativos e informativos.
- Alguns textos de Fundamental I e II começam com o caractere de marca de ordem de bytes (BOM, U+FEFF), mantido porque a leitura usou encoding='utf-8' e não utf-8-sig. A interface remove o BOM (utf-8-sig), mas o efeito nas features é desprezível: no máximo um token a menos em centenas.

**Implicação:** o classificador pode aprender, em parte, assinaturas de fonte e formatação em vez de complexidade linguística (*shortcut learning*). Esse risco não foi testado e é discutido como limitação na Seção 7.

# 3. Metodologia

### 3.1 Visão geral das 30 features

| Grupo | Nº | Features | Ferramenta |
|---|---|---|---|
| Pontuação | 10 | punct_virgula, punct_ponto_final, punct_reticencias, punct_interrogacao, punct_exclamacao, punct_ponto_virgula, punct_dois_pontos, punct_aspas, punct_parenteses, punct_travessao | Regex |
| Sentenças | 2 | sent_tam_medio, sent_tam_desvio | spaCy |
| Classes gramaticais | 13 | pos_NOUN, pos_VERB, pos_ADJ, pos_ADV, pos_PRON, pos_DET, pos_ADP, pos_CCONJ, pos_SCONJ, pos_AUX, pos_NUM, pos_PROPN, pos_INTJ | spaCy |
| Léxicas | 4 | lex_tam_medio_palavra, lex_desvio_tam_palavra, lex_prop_palavras_longas, lex_ttr_300 | spaCy |
| Canonicidade | 1 | canonicidade | NLTK + regex |
| **Total** | **30** | | |

A função orquestradora extrair_features_nlp(texto) recebe **um texto** e devolve um dicionário nome → valor, cuja ordem é fixada pela lista NOMES_FEATURES. A ordem e a quantidade importam: o modelo é treinado com um vetor de 30 posições, e qualquer divergência na interface geraria erro ou, pior, previsões silenciosamente erradas. Se o texto não tiver nenhuma palavra, a função devolve o vetor zerado com as mesmas chaves, para não quebrar o DataFrame.

O desenho é de **uma função por texto, aplicada ao DataFrame fora dela** (adicionar_features). Isso mantém a função testável isoladamente e reutilizável na interface com um texto novo. Como a extração é uma regra fixa, sem parâmetros aprendidos, calculá-la nos três conjuntos não causa vazamento de dados. A função adicionar_features descarta as colunas de feature já existentes antes de recalcular, o que a torna idempotente: pode ser reexecutada após qualquer mudança de feature sem duplicar colunas.

### 3.2 Processamento comum

- **spaCy (pt_core_news_sm, NER desativado).** O pipeline (tokenização, etiquetagem gramatical e parser de dependências) roda uma única vez por texto, e o objeto doc é reaproveitado em todas as features baseadas em spaCy.
- **Palavras.** São os tokens com is_alpha verdadeiro, o que exclui pontuação, números e símbolos. O total n_palavras é o denominador de todas as proporções.

- **Por que proporções.** Contagens brutas dependem do tamanho do texto: um texto longo teria mais vírgulas que um curto, e o modelo aprenderia o tamanho e não o estilo. Dividir por n_palavras torna textos de tamanhos diferentes comparáveis (ver Seção 2.5).
- **spaCy e NLTK.** O etiquetador padrão do NLTK (averaged_perceptron_tagger) é treinado em inglês; para português seria preciso treiná-lo no corpus Mac-Morpho. O spaCy já traz um etiquetador em português com etiquetas universais. Por isso a solução é híbrida: spaCy nas features de sentenças, classes gramaticais e léxicas; NLTK (word_tokenize) dentro da canonicidade. O custo é tokenizar o texto duas vezes.

### 3.3 Pontuação

Cada sinal é contado com uma expressão regular aplicada ao **texto cru** (a regex dá controle total, sem depender de como o spaCy agrupa os sinais) e dividido por n_palavras:

- vírgula: ,
- ponto final: (?<!\.)\.(?!\.), ou seja, um ponto que não é precedido nem seguido de outro ponto; sem esse cuidado, cada reticência somaria três pontos finais;
- reticências: três pontos consecutivos ou o caractere …;
- interrogação ?, exclamação !, ponto e vírgula ;, dois pontos :;
- aspas: retas e tipográficas (", “, ”, «, »);
- parênteses: ( e );
- travessão: — e –.

**Limitação:** pontos de abreviações (Sr.) e de números decimais também entram na contagem de ponto final. A barra (/) não é contada, embora apareça em cabeçalhos de alguns textos (Seção 2.6). É uma aproximação aceitável para uma feature estatística.

### 3.4 Tamanho das sentenças

O spaCy segmenta o texto em sentenças (doc.sents) usando o parser sintático, mais robusto que quebrar em . (lida melhor com abreviações e reticências). Para cada sentença, contam-se as palavras (is_alpha); sentenças sem nenhuma palavra são descartadas. As features são a **média** (sent_tam_medio) e o **desvio-padrão populacional** (sent_tam_desvio, np.std com ddof=0) desses tamanhos. A hipótese é que sentenças mais longas e mais variadas indicam construções sintáticas mais elaboradas (subordinação, orações intercaladas).

### 3.5 Classes gramaticais

Cada palavra recebe uma etiqueta universal do spaCy (token.pos_). Calcula-se a proporção de cada uma das 13 classes da lista fixa (NOUN, VERB, ADJ, ADV, PRON, DET, ADP, CCONJ, SCONJ, AUX, NUM, PROPN, INTJ). Três pontos de projeto:

- o **placar inicializado com todas as classes em zero** garante que todo texto gere as mesmas colunas, na mesma ordem;
- classes fora da lista (PART, SYM, X) são ignoradas, então a soma das 13 proporções fica próxima de 1, mas pode ficar um pouco abaixo;
- os erros de etiquetagem do modelo pequeno (sm) se propagam para a feature, principalmente em textos com cabeçalhos e fragmentos.

Hipótese inicial de direção com a complexidade: mais substantivos, adjetivos e preposições nos níveis altos; mais verbos e pronomes nos níveis baixos.

### 3.6 Features lexicais

Todas operam sobre as formas das palavras em minúsculas (is_alpha), sem lematização:

| Feature | Definição |
|---|---|
| lex_tam_medio_palavra | Média do número de letras por palavra |
| lex_desvio_tam_palavra | Desvio-padrão do número de letras por palavra (variedade entre palavras curtas e longas) |
| lex_prop_palavras_longas | Proporção de palavras com **mais de 6 letras** (limiar inicial, não ajustado) |
| lex_ttr_300 | *Type-token ratio* nas **primeiras 300 palavras**: palavras distintas ÷ palavras da janela |

O TTR clássico diminui conforme o texto cresce, pois palavras se repetem. Como as classes altas têm textos mais longos (Seção 2.5), um TTR sobre o texto inteiro teria um viés: o Superior pareceria ter vocabulário menos rico só por ser mais longo. A **janela fixa de 300 palavras** (o piso comum do corpus) mede todos os textos no mesmo comprimento. Para textos com menos de 300 palavras alfabéticas, usa-se o que houver.

### 3.7 Canonicidade (feature obrigatória)

**Definição.** Uma palavra é canônica quando é formada exclusivamente por sílabas do tipo consoante + vogal (CV), como em *ja-ne-la* ou *ga-to*. A feature é a **proporção de palavras canônicas** do texto. A silába canônica é CV em termos de **som**, e não de letra; por isso a implementação trata os dígrafos antes da checagem.

**Algoritmo (calcular_canonicidade):**

1. converte para minúsculas e normaliza o Unicode para a forma composta (NFC), de modo que ã seja um único caractere e não a + til solto;

2. tokeniza com word_tokenize(..., language='portuguese') do NLTK e mantém apenas tokens alfabéticos (isalpha), descartando pontuação e números;
3. para cada palavra, substitui cada dígrafo por uma consoante marcadora (b);
4. aplica as três regras: (R1) o tamanho deve ser par; (R2) nas posições pares (0, 2, 4...) a letra deve ser consoante; (R3) nas posições ímpares a letra deve ser vogal;
5. devolve palavras canônicas ÷ total de palavras (0,0 se não houver palavras, evitando divisão por zero).

**Dígrafos tratados:**

ch | lh | nh | rr | ss | qu(?=[eéêií]) | gu(?=[eéêií])

ch, lh, nh, rr e ss representam um único som consonantal. qu e gu só contam como dígrafo antes de e ou i (que-ro, gue-rra); antes de a ou o (quando, guardar) o u é pronunciado e a palavra não é tratada como canônica. O (?=...) é um lookahead: confere a letra seguinte sem consumi-la.

**Validação por casos de teste** (saídas do notebook):

| Entrada | Resultado | Explicação |
|---|---|---|
| eu | 0,0 | Ditongo, não é CV (antes da correção, o resultado era 1,0) |
| chave | 1,0 | ch conta como uma consoante: cha-ve |
| carro | 1,0 | rr conta como uma consoante: ca-rro |
| campo | 0,0 | Sílaba CVC (cam-po) |
| Em 2024 a janela abriu. | 0,25 | Entram 4 palavras (o número fica de fora); só janela é canônica |
| A janela estava aberta para a sopa esfriar. | 0,375 | 3 de 8: janela, para, sopa |

**Limitações conhecidas:** (i) aproxima a estrutura fonética pela ortografia (a letra h muda, por exemplo, conta como consoante); (ii) encontros como br, pl, sc e ditongos como ei e ão não são CV, o que é coerente com a definição; (iii) a medida é **rígida**: uma única sílaba não canônica torna a palavra inteira não canônica (*cantar* = can-tar é não canônica mesmo tendo uma sílaba CV). Uma alternativa mais fina, a proporção de **sílabas** canônicas, exige separação silábica real e fica como trabalho futuro.

### 3.8 Protocolo de avaliação

- **Validação simples** (309 textos) para decisões rápidas de engenharia de features. Como a amostra é pequena (44 e 49 textos nas classes menores), diferenças de 1 a 2 pontos são ruído.
- **Validação cruzada estratificada de 5 dobras** (StratifiedKFold, shuffle=True, random_state=42) sobre treino + validação (1.751 textos). As **mesmas dobras** foram usadas em todas as comparações, o que torna a comparação pareada.

- **Métrica de decisão:** F1 macro. **Regra de decisão:** uma mudança só conta como melhora se a média da validação cruzada subir mais do que o desvio típico (≈ 0,016 a 0,02); dentro dessa faixa, considera-se empate.
- **Prevenção de vazamento:** os escaladores (StandardScaler, MinMaxScaler) aprendem média e desvio dos dados, ao contrário da extração de features, que é uma regra fixa. Por isso foram encapsulados em Pipeline (make_pipeline): em cada dobra, o escalador é ajustado só nas partes de treino e apenas aplicado à parte de validação.
- **Teste:** usado **uma única vez**, ao final, com o modelo treinado em treino + validação.

### 3.9 Algoritmos comparados

| Algoritmo | Configuração | Pré-processamento | Observações |
|---|---|---|---|
| Random Forest | n_estimators=300, class_weight='balanced', random_state=42 | Nenhum | Insensível à escala; fornece importância das features |
| SVC | Padrão do scikit-learn (kernel RBF, C=1, gamma='scale'), class_weight='balanced' | StandardScaler | Muito sensível à escala; fronteira não linear |
| Regressão Logística | max_iter=2000, class_weight='balanced' | StandardScaler | Fronteiras lineares |
| MultinomialNB | Padrão | MinMaxScaler | Exige valores não negativos (o StandardScaler geraria negativos); não tem class_weight; pensado para contagens e TF-IDF |

Os hiperparâmetros ficaram nos valores padrão (exceto o balanceamento); não houve busca em grade.

# 4. Experimentos e resultados

Esta seção relata, em ordem cronológica, as etapas do desenvolvimento e a decisão tomada a partir de cada resultado, incluindo os erros encontrados e os experimentos descartados. Todos os números das Seções 4.2 a 4.8 vêm do conjunto de validação ou da validação cruzada; o conjunto de teste só é usado na Seção 5.

### 4.1 Resumo cronológico

| # | Experimento | Resultado (F1 macro) | Decisão |
|---|---|---|---|
| 1 | Random Forest, 26 features, sem balanceamento | 0,697 (validação) | Ponto de partida |
| 2 | class_weight='balanced' | 0,701 (validação); recall de Fund. II: 0,45 → 0,61 | Mantido |
| 3 | + 4 features lexicais (30 features) | 0,766 (validação); 0,771 ± 0,016 (CV) | Mantidas |
| 4 | + 4 features de riqueza lexical (34 features) | 0,745 (validação); 0,761 ± 0,019 (CV) | Descartadas |
| 5 | Comparação de modelos (CV, 30 features) | SVC 0,785; RF 0,771; Reg. Logística 0,742; MultinomialNB 0,342 | SVC escolhido |

### 4.2 Desenvolvimento e correção da canonicidade

A primeira versão da função calcular_canonicidade tinha lógica correta de alternância consoante–vogal, mas apresentou problemas identificados na revisão do código e corrigidos antes de qualquer experimento:

| # | Problema | Efeito | Correção |
|---|---|---|---|
| 1 | Dígrafos tratados como duas consoantes | chave, galinha, carro e guerra eram classificadas como não canônicas, embora sejam CV em termos de som | Substituição dos dígrafos por uma consoante marcadora **antes** da regra de tamanho, pois eles mudam o comprimento (chave tem 5 letras e vira 4) |
| 2 | isalnum() aceitava tokens numéricos | 2024 entrava no denominador e nunca podia ser canônico, distorcendo a proporção | Troca por isalpha() |
| 3 | Conjuntos de vogais/consoantes recriados a cada chamada; acentos decompostos; faltava à | Custo desnecessário; a + til solto não casava com nenhum conjunto | Constantes no nível do módulo; normalização NFC; à adicionada |
| 4 | nltk.download repetido dentro do trecho da função | Chamadas de rede desnecessárias, mas foi utilizado com fim didático | Downloads movidos para o início do módulo |

Por decisão de projeto, as correções preservaram a estrutura original da função (tokenização, laço com as Regras 1, 2 e 3, retorno da proporção). Uma reescrita compacta baseada em expressão regular (fullmatch de (CV)+) foi considerada e descartada para não alterar a estrutura.

Também ficou estabelecido que a função recebe **um texto**, e não o DataFrame. A canonicidade é uma regra fixa escrita pelo autor: o modelo não aprende a calculá-la, apenas recebe o número já calculado e aprende a relação entre esse número e a categoria. Receber um texto torna a função testável e reutilizável na predição de textos novos.

### 4.3 Análise descritiva das 26 features iniciais

Antes de treinar qualquer modelo, calculou-se a média de cada feature por categoria no conjunto de treino. A criação de uma feature é uma aposta; a tabela é o teste dessa aposta.

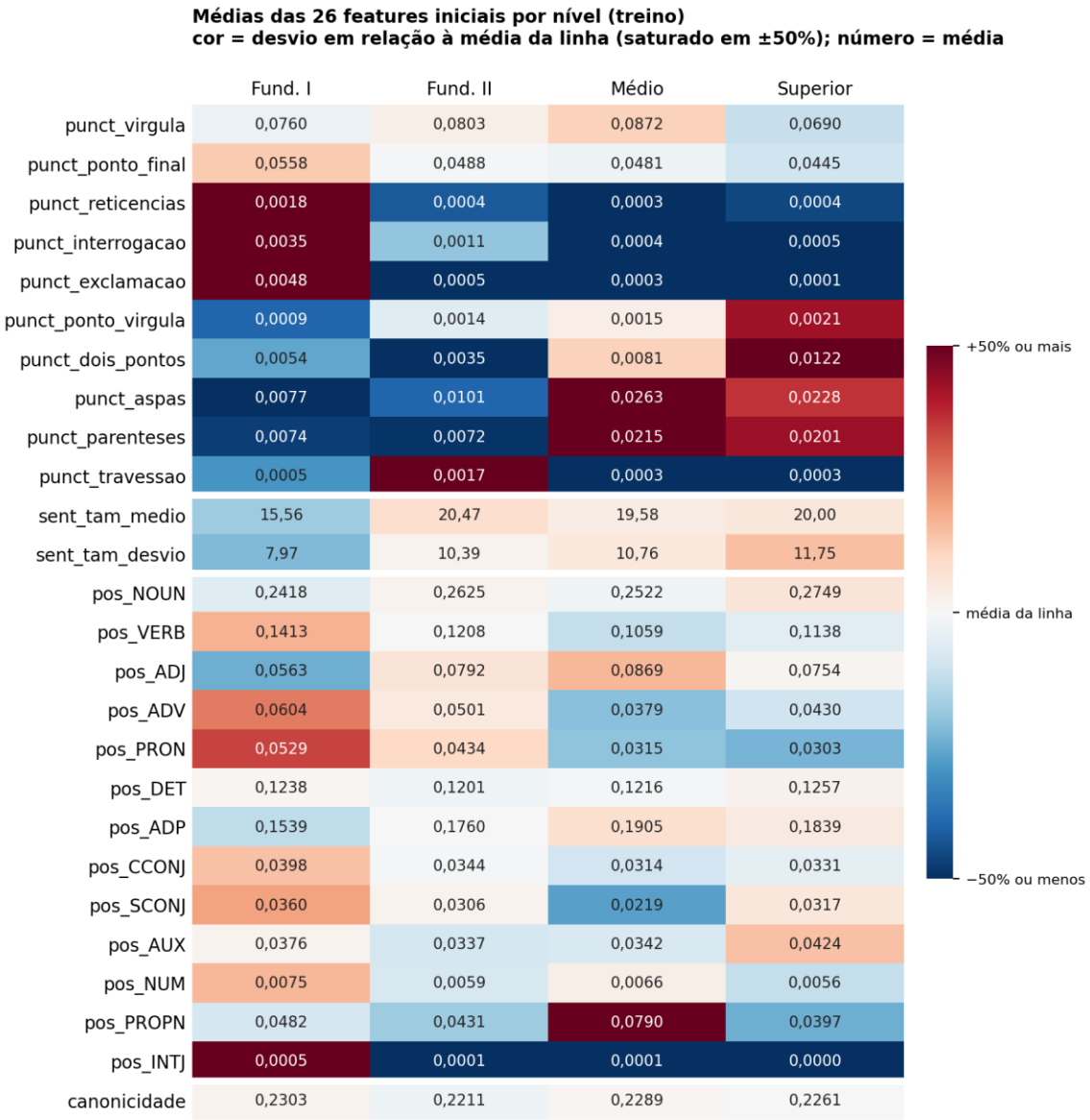



***Figura 3** — Médias das 26 features iniciais por nível, no conjunto de treino (substitui a tabela de médias). A cor mostra o desvio de cada média em relação à média da linha, saturado em ±50%; o número mostra a média. Desvios relativos amplificam features de valores muito pequenos, como as de pontuação rara, então a cor indica a direção da diferença e não a sua importância para o modelo.*

**Leitura:**

- **Fundamental I se separa das demais.** sent_tam_medio salta de 15,6 para 20,5 e depois estabiliza (19,6 e 20,0). pos_PRON cai de forma monotônica (0,053 → 0,043 → 0,032 → 0,030); pos_VERB e pos_ADV caem de Fund. I a Médio; pos_ADJ e pos_ADP sobem até o Médio; exclamações caem de 0,0048 para 0,0001 e interrogações de 0,0035 para 0,0004.

- **Fund. II, Médio e Superior são parecidos entre si** na maioria das features (por exemplo, pos_DET entre 0,120 e 0,126 e sent_tam_medio entre 19,6 e 20,5). O diagnóstico antecipou que essas três classes seriam as mais difíceis de separar.
- **Sinais de nível alto:** sent_tam_desvio sobe monotonicamente (7,97 → 11,75); punct_ponto_virgula e punct_dois_pontos crescem até o Superior; punct_aspas e punct_parenteses saltam no Médio (aspas de 0,010 para 0,026 e parênteses de 0,007 para 0,022, de Fund. II para Médio).
- **pos_PROPN não segue tendência de complexidade:** 0,048, 0,043, 0,079, 0,040. O pico no Médio sugere influência do conteúdo ou do formato dos textos, e não do nível de linguagem (hipótese; ver Seção 7).
- **canonicidade é praticamente constante** (0,221 a 0,230, variação de cerca de 4% em torno de 0,226) e sem tendência. Hipótese: a medida conta palavras *inteiramente* CV, em geral palavras funcionais curtas e frequentes (*para*, *uma*, *dele*), presentes em qualquer nível, o que dilui o sinal.

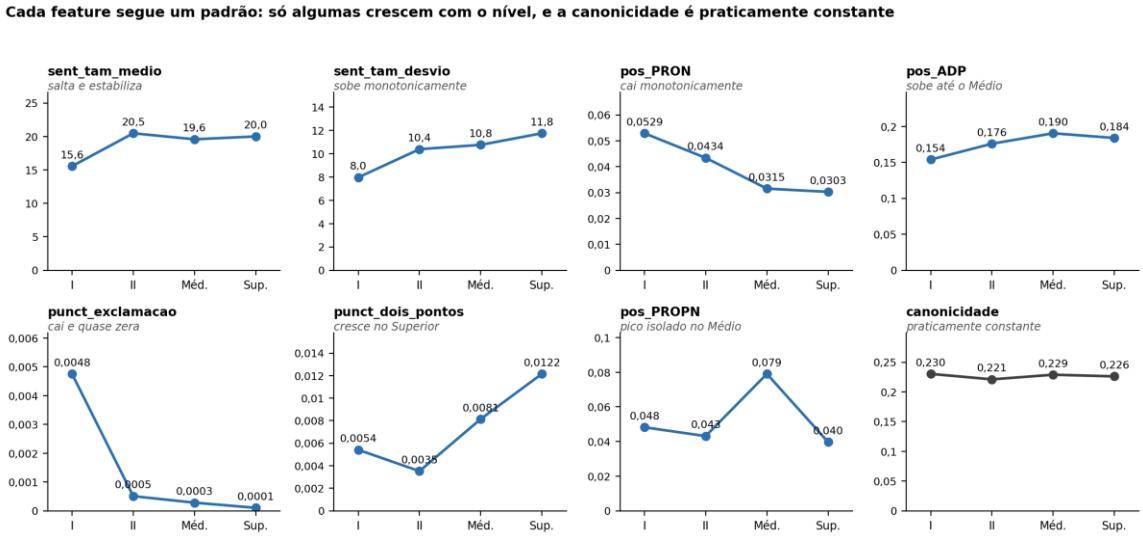



***Figura 4*** *— Perfil de oito features por nível, com os mesmos valores da Figura 3. O eixo y parte de zero em todos os painéis, para não exagerar diferenças pequenas (como na canonicidade).*

### 4.4 Baseline: Random Forest com 26 features e balanceamento de classes

O Random Forest foi escolhido para o primeiro modelo porque não exige padronização (as features têm escalas muito diferentes, como sent_tam_medio ≈ 20 e punct_travessao ≈ 0,001) e porque informa a importância de cada feature.

| Configuração | Acurácia | F1 macro | Recall Fund. I | Recall Fund. II | Recall Médio | Recall Superior |
|---|---|---|---|---|---|---|
| Sem balanceamento | 0,718 | 0,697 | 0,91 | 0,45 | 0,62 | 0,84 |
| class_weight='balanced' | 0,712 | 0,701 | 0,91 | 0,61 | 0,59 | 0,78 |

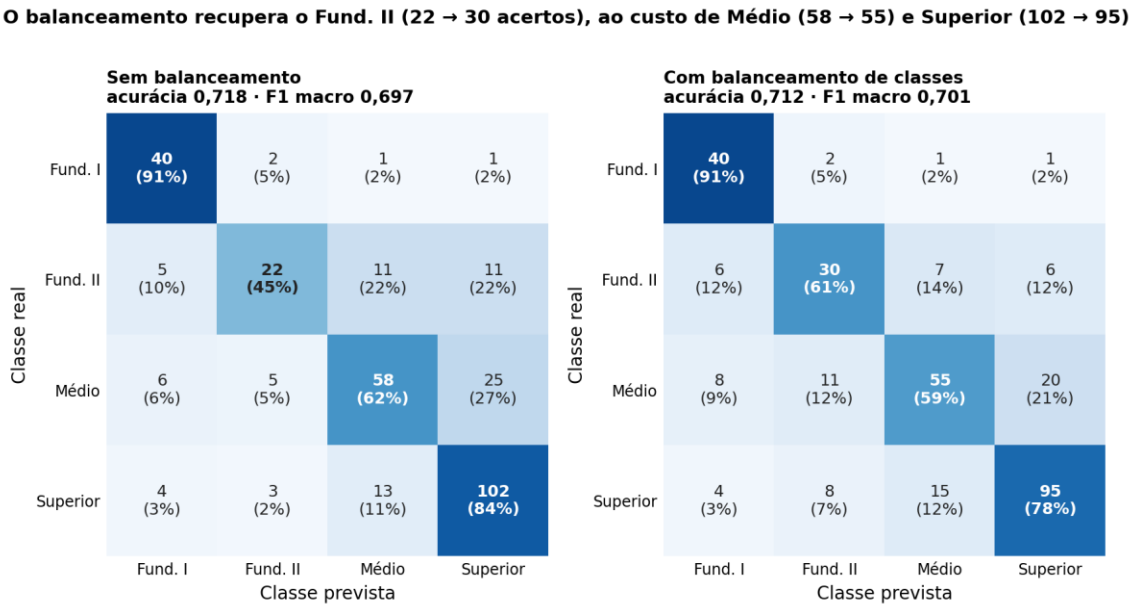



***Figura 5*** *— Matrizes de confusão do Random Forest com 26 features na validação (substituem as tabelas de confusão). Linhas = classe real; colunas = classe prevista; cada célula mostra a contagem e a porcentagem da linha, de modo que a diagonal é o recall. Esquerda: sem balanceamento; direita: com* class_weight='balanced'.

**Interpretação do baseline:**

- 71,8% de acurácia está bem acima dos pisos (25% e 39,5%) para um modelo com features puramente estruturais.
- Os erros se concentram em **classes vizinhas**: o modelo capturou uma ordem de complexidade. Fund. I é bem identificado (recall 0,91), e Fund. II é « engolido » pelas classes acima (11 textos para Médio e 11 para Superior).
- As importâncias são **planas**: a maior (pos_NOUN) vale apenas 0,065 e quase todas ficam entre 0,03 e 0,07. Nenhuma feature resolve sozinha a separação. pos_PROPN aparece em terceiro (0,062), apesar de sua média não seguir a complexidade; canonicidade fica em 19º de 26 (0,029).

**Efeito do balanceamento:** o F1 macro praticamente não se moveu (+0,004, dentro do ruído), mas a distribuição dos acertos mudou. O recall de Fund. II subiu de 0,45 para 0,61 (22 → 30 acertos), ao custo da precision de Fund. II (0,69 → 0,59) e de perdas em Médio e Superior: o balanceamento **trocou** acertos de classes grandes por acertos de uma classe pequena. A conclusão foi que **o gargalo era a falta de informação nas features, e não o desbalanceamento**. Decisão: manter class_weight='balanced', por equilibrar o desempenho entre as classes sem custo em F1 macro.

### 4.5 Features lexicais: de 26 para 30 features

Com as 26 features, Fund. II, Médio e Superior continuavam misturados. A hipótese foi que a complexidade lexical (palavras mais longas e vocabulário mais variado) traria informação nova. Antes de implementar, verificou-se o tamanho dos textos (Seção 2.5), o que levou ao TTR em janela fixa de 300 palavras. Médias das quatro features novas no treino:

| Feature | Fund. I | Fund. II | Médio | Superior |
|---|---|---|---|---|
| lex_tam_medio_palavra | 4,683 | 5,135 | 5,048 | 5,104 |
| lex_desvio_tam_palavra | 2,751 | 3,162 | 3,127 | 3,276 |
| lex_prop_palavras_longas | 0,249 | 0,323 | 0,319 | 0,333 |
| lex_ttr_300 | 0,608 | 0,597 | 0,544 | 0,501 |

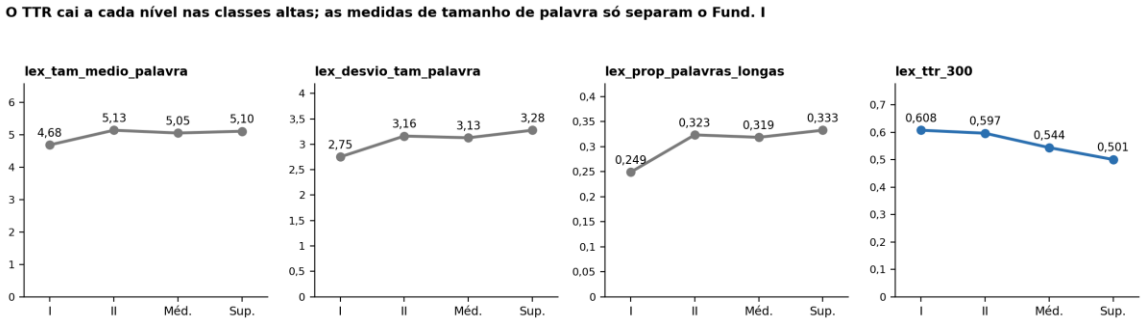



***Figura 6** — Médias das quatro features lexicais por nível, no treino (eixo y a partir de zero). Em azul, o TTR em 300 palavras; em cinza, as três medidas de tamanho de palavra.*

As três features de tamanho de palavra têm o mesmo perfil de sent_tam_medio: um salto de Fund. I para as demais e depois estabiliza. O **lex_ttr_300 é o comportamento mais valioso**: cai de forma monotônica nas três classes mais altas (0,597 → 0,544 → 0,501), justamente onde as outras features falham. A direção é contrária à hipótese inicial (vocabulário « mais rico » nos níveis altos). Uma explicação plausível, **não verificada**, é que textos de nível alto são mais coesos e repetem termos temáticos, enquanto textos infantis variam mais as palavras em frases curtas.

Resultado do Random Forest balanceado com 30 features, na validação:

| Métrica | 26 features | 30 features | Variação |
|---|---|---|---|
| Acurácia | 0,712 | 0,770 | +0,058 |
| F1 macro | 0,701 | 0,766 | +0,065 |
| Recall Fund. I | 0,91 | 0,98 | +0,07 |
| Recall Fund. II | 0,61 | 0,69 | +0,08 |
| Recall Médio | 0,59 | 0,60 | +0,01 |
| Recall Superior | 0,78 | 0,86 | +0,08 |
| Precision Fund. II | 0,59 | 0,69 | +0,10 |

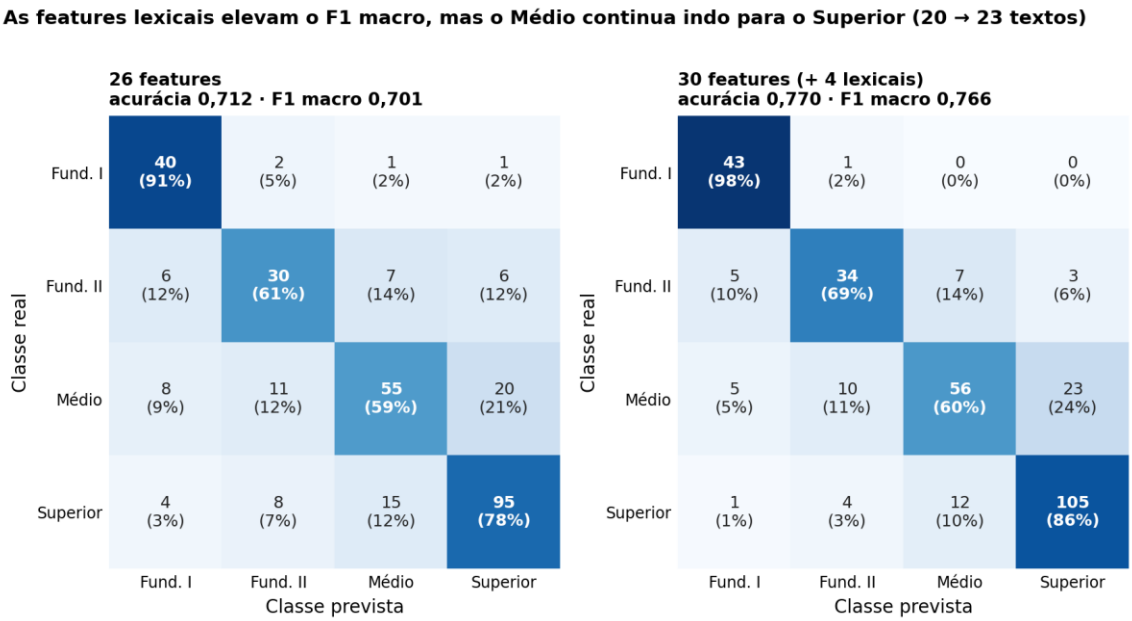



***Figura 7*** *— Matrizes de confusão do Random Forest balanceado na validação, com 26 e com 30 features (substituem a tabela de confusão). Cada célula mostra a contagem e a porcentagem da linha.*

**Interpretação:**

- Diferentemente do balanceamento, o ganho de 6,5 pontos de F1 macro está acima do que se atribuiria a ruído. Fund. II ganhou recall **e** precision ao mesmo tempo, sem o *trade-off* do experimento anterior.
- **O Médio continua sendo o gargalo:** recall de 0,60 e 23 textos de Médio classificados como Superior (eram 20). A confusão Médio ↔ Superior soma 35 erros (23 + 12).
- **Importâncias:** lex_ttr_300 lidera com 0,101, cerca de 1,5 vez a maior importância do baseline (0,065), seguida de lex_desvio_tam_palavra (0,081), lex_prop_palavras_longas (0,055) e lex_tam_medio_palavra (0,047). O segundo lugar do desvio do tamanho de palavra surpreende, dado que sua média é quase igual entre Fund. II, Médio e Superior; provavelmente ele separa o Fund. I, e a importância é inflada pela correlação com as outras duas medidas de comprimento. Não deve ser lido como « a segunda melhor feature » sem teste adicional.
- **canonicidade caiu para 0,019** (24ª de 30).

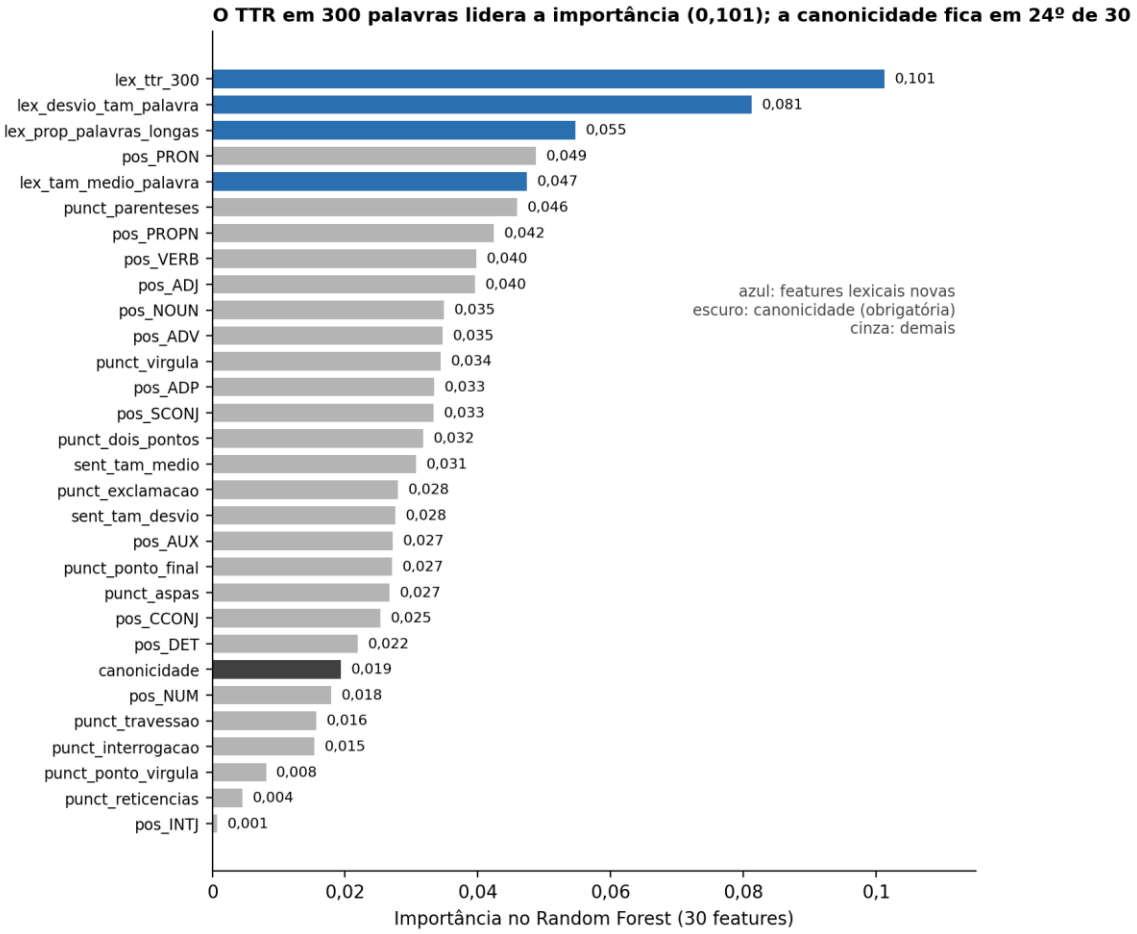



***Figura 8*** *— Importância das 30 features no Random Forest balanceado (validação). Azul: features lexicais novas; escuro: canonicidade (feature obrigatória); cinza: demais. A importância é um guia, e não um veredito, porque várias features são correlacionadas.*

### 4.6 Validação cruzada: o ganho é real, mas menor que o aparente

A validação simples tem só 309 textos (44 e 49 nas classes menores). Para obter uma estimativa estável, foi usada a validação cruzada estratificada de 5 dobras sobre treino + validação (1.751 textos), com as mesmas dobras para todas as configurações.

| Configuração (Random Forest balanceado) | F1 macro, validação simples | F1 macro, CV de 5 dobras |
|---|---|---|
| 26 features | 0,701 | 0,740 ± 0,029 |
| 30 features | 0,766 | **0,771 ± 0,016** |
| 34 features | 0,745 | 0,761 ± 0,019 |

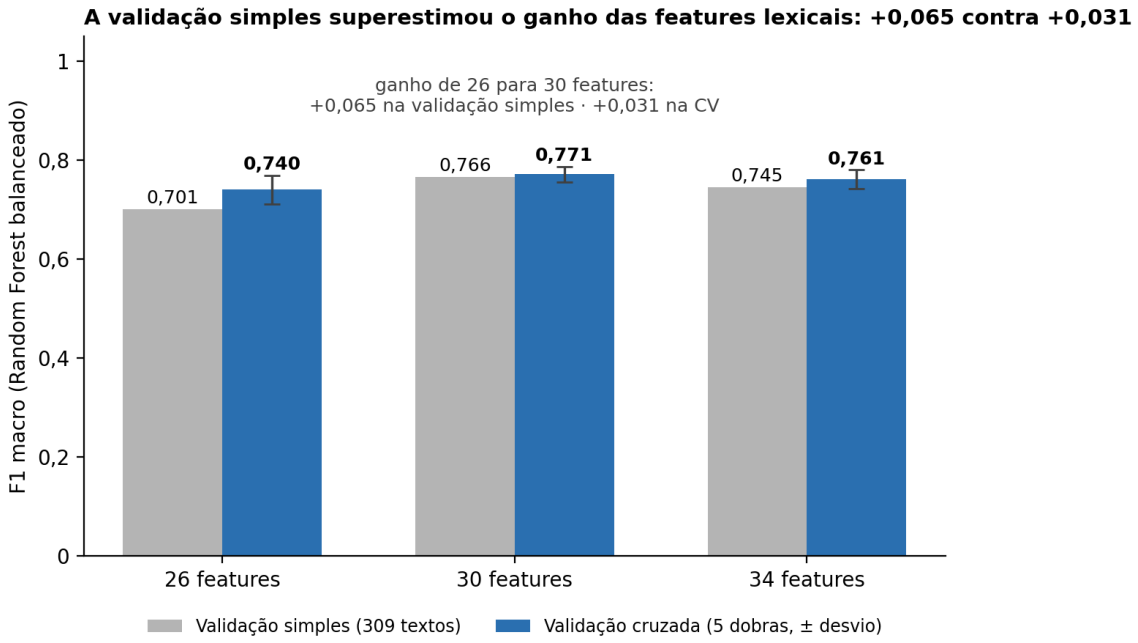



***Figura 9** — F1 macro do Random Forest balanceado por conjunto de features: validação simples (309 textos) e validação cruzada de 5 dobras (barra de erro = desvio-padrão). Fonte: Seções 4.5 e 4.6.*

F1 macro por dobra com 30 features: 0,780; 0,778; 0,766; 0,788; 0,742.

- O ganho de 26 para 30 features na CV é de **+0,031**, cerca de duas vezes o desvio de cada configuração, e a comparação é pareada (mesmas dobras).
- A validação simples **superestimou o ganho** (+0,065), porque subestimou as 26 features (0,701 contra 0,740 na CV). O número honesto para reportar é **+3 pontos de F1 macro**.
- O desvio caiu de 0,029 para 0,016: o modelo também ficou mais estável.
- A partir daqui, uma mudança só é considerada melhora se a média subir mais do que o desvio (≈ 0,016).

**Decisão:** manter as 30 features. A canonicidade permanece no vetor por ser requisito do projeto, mesmo com efeito mínimo; por isso, a ablação dessa feature foi cancelada.

### 4.7 Features de riqueza lexical: de 30 para 34 (resultado negativo)

Como o TTR foi a melhor contribuição, testou-se a mesma família com medidas mais robustas, visando o gargalo Médio ↔ Superior:

- lex_mattr_50: média do TTR em todas as janelas móveis de 50 palavras (menos sensível a repetições pontuais);
- lex_prop_hapax: palavras de frequência 1 ÷ palavras distintas, nas primeiras 300;
- lex_ttr_lemas_300: TTR sobre lemas (exigiu reativar o lematizador do spaCy);

- lex_entropia_300: entropia de Shannon normalizada das frequências, nas primeiras 300 palavras.

**Resultados com 34 features:**

| Avaliação | 30 features | 34 features |
|---|---|---|
| Validação: acurácia / F1 macro | 0,770 / 0,766 | 0,748 / 0,745 |
| Validação: recall do Médio | 0,60 | 0,56 |
| Validação: Médio classificado como Superior | 23 | 26 |
| CV (F1 macro) | 0,771 ± 0,016 | 0,761 ± 0,019 |
| CV por dobra | 0,780; 0,778; 0,766; 0,788; 0,742 | 0,773; 0,764; 0,752; 0,787; 0,731 |

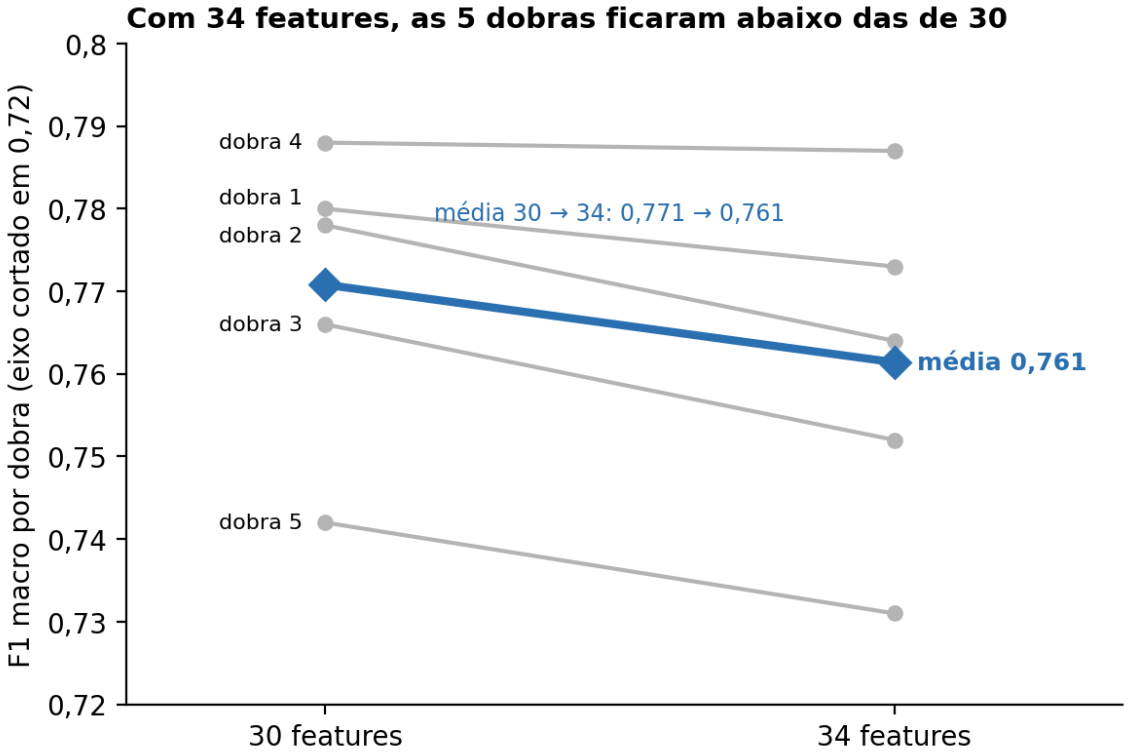



***Figura 10*** *— F1 macro em cada uma das 5 dobras com 30 e com 34 features (mesmas dobras). O eixo y não parte de zero (está cortado em 0,72) para tornar visíveis as diferenças entre dobras; as cinco linhas descem.*

A queda de 0,010 na CV é menor que o desvio, ou seja, **empate estatístico**. Porém, as cinco dobras ficaram abaixo das anteriores, o que indica uma piora leve e consistente, de pequena magnitude. O alvo do experimento não melhorou: o recall do Médio caiu.

**Interpretação (hipóteses, não separadas experimentalmente):**

1. **Redundância.** lex_mattr_50, lex_ttr_lemas_300 e lex_ttr_300 medem quase a mesma coisa (diversidade de vocabulário). A importância se dividiu entre elas (0,067, 0,054 e 0,053), enquanto o TTR sozinho tinha 0,101.
2. **Ruído.** Com ≈ 1.700 textos, cada coluna redundante dá às árvores mais chances de dividir por flutuações casuais.
3. **O gargalo pode não ser de riqueza lexical.** Médio e Superior podem ter riqueza parecida e diferir em tema, vocabulário técnico ou sintaxe.

**Decisão:** descartar as quatro features e voltar ao vetor de 30. A ablação individual de cada feature nova foi proposta, mas não executada. Por restrição de prazo, não foram exploradas novas features voltadas ao gargalo Médio ↔ Superior; as direções possíveis estão na Seção 8. Registrar este resultado negativo é relevante:  hipótese testada com critério e rejeitada.

### 4.8 Comparação dos quatro algoritmos (CV, 30 features)

| Modelo | F1 macro | Desvio | Acurácia |
|---|---|---|---|
| SVC | 0,785 | 0,015 | 0,790 |
| Random Forest | 0,771 | 0,016 | 0,778 |
| Regressão Logística | 0,742 | 0,019 | 0,741 |
| MultinomialNB | 0,342 | 0,017 | 0,487 |

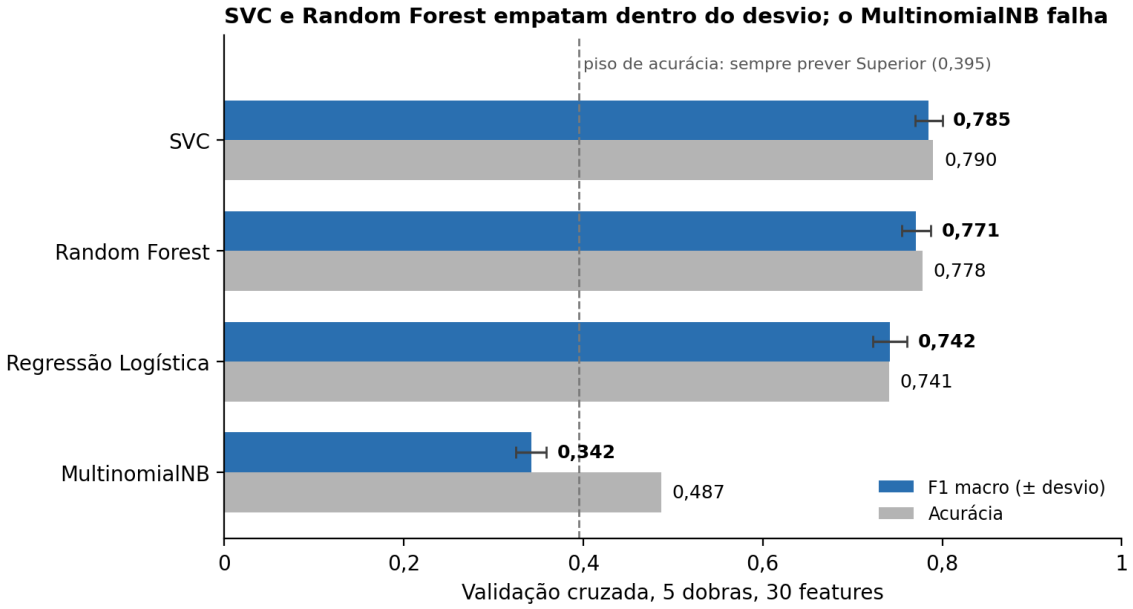



***Figura 11*** *— Comparação dos quatro algoritmos na validação cruzada de 5 dobras, com 30 features: F1 macro (± desvio) e acurácia. A linha tracejada marca o piso de acurácia de sempre prever a classe majoritária (0,395).*

- **SVC e Random Forest empatam tecnicamente:** a diferença de 0,014 é menor que o desvio (≈ 0,015). O SVC lidera na média, no desvio e na acurácia, mas sem margem para afirmar superioridade.
- **A Regressão Logística fica ≈ 0,04 atrás do SVC.** Hipótese: ela traça fronteiras lineares, e a relação entre as features e as classes provavelmente não é linear; o SVC com kernel RBF e o Random Forest capturam essas curvas.
- **O MultinomialNB falha** (F1 macro 0,342; acurácia 0,487, pouco acima do piso de 39,5%), compatível com um modelo que praticamente ignora as classes menores. A causa é inadequação ao tipo de dado, e não erro de código: o modelo assume que as features são **contagens** e multiplica probabilidades por feature, mas as nossas são proporções contínuas, que o `MinMaxScaler` comprime em 0 a 1; além disso, não tem `class_weight`. Ele só deve ser competitivo com contagens de palavras ou TF-IDF.

### 4.9 Escolha do modelo final

O **SVC** foi escolhido por ter a melhor média de F1 macro, o menor desvio e a melhor acurácia na CV. Ressalvas: (i) SVC e Random Forest empatam tecnicamente; a comparação pareada por dobra (em quantas das 5 dobras o SVC vence) foi proposta e não executada; o Random Forest permanece como alternativa interpretável, por fornecer importâncias de features. (ii) Os hiperparâmetros ficaram nos valores padrão; a busca em grade de C e gamma (GridSearchCV) não foi executada, e seu melhor escore, por ser escolhido nas mesmas dobras, seria levemente otimista.

### 4.10 Registro consolidado de problemas e correções

| Etapa | Problema | Correção ou decisão |
|---|---|---|
| Dados | 16 textos duplicados (risco de vazamento treino/teste) | Removidos antes da divisão |
| Canonicidade | Letra e em CONSOANTES (eu = 1,0) | Removida |
| Canonicidade | Dígrafos contados como duas consoantes | Substituição por consoante marcadora |
| Canonicidade | isalnum() incluía números no denominador | isalpha() |
| Features | Recalcular as features duplicaria colunas | drop idempotente em adicionar_features |
| Avaliação | Validação simples (309 textos) superestimou o ganho das features lexicais (+0,065 contra +0,031 na CV) | Validação cruzada pareada e regra de decisão pelo desvio |
| Experimento | 34 features sem ganho (redundância) | Descartadas |
| Modelo | MultinomialNB inadequado para proporções contínuas | Documentado; sem balanceamento possível |
| Escopo | Gargalo Médio ↔ Superior não resolvido | Registrado como limitação (Seções 7 e 8) |

# 5. Resultado final (teste)

## 5.1 Procedimento

O modelo final é um Pipeline com StandardScaler e SVC(class_weight='balanced', random_state=42), treinado em treino + validação (1.751 textos). O conjunto de teste (309 textos, nunca usado em nenhuma decisão) foi avaliado uma única vez, depois de definidos features, balanceamento e algoritmo. O pipeline treinado foi salvo em modelo_svc_complexidade.joblib.

## 5.2 Resultados

**Acurácia 0,790 e F1 macro 0,778**, consistentes com a validação cruzada (0,785 ± 0,015): a diferença de 0,007 é menor que o desvio. O modelo generaliza e a escolha do SVC não foi otimista. Ambos os valores estão bem acima dos pisos de 25% e 39,5%.

| Classe | Precision | Recall | F1 | Suporte |
|---|---|---|---|---|
| Fundamental I | 0,80 | 0,91 | 0,85 | 44 |
| Fundamental II | 0,68 | 0,65 | 0,67 | 49 |
| Médio | 0,79 | 0,72 | 0,76 | 94 |
| Superior | 0,83 | 0,85 | 0,84 | 122 |
| Média macro | 0,77 | 0,78 | 0,78 | 309 |
| Média ponderada | 0,79 | 0,79 | 0,79 | 309 |

## 5.3 Matriz de confusão

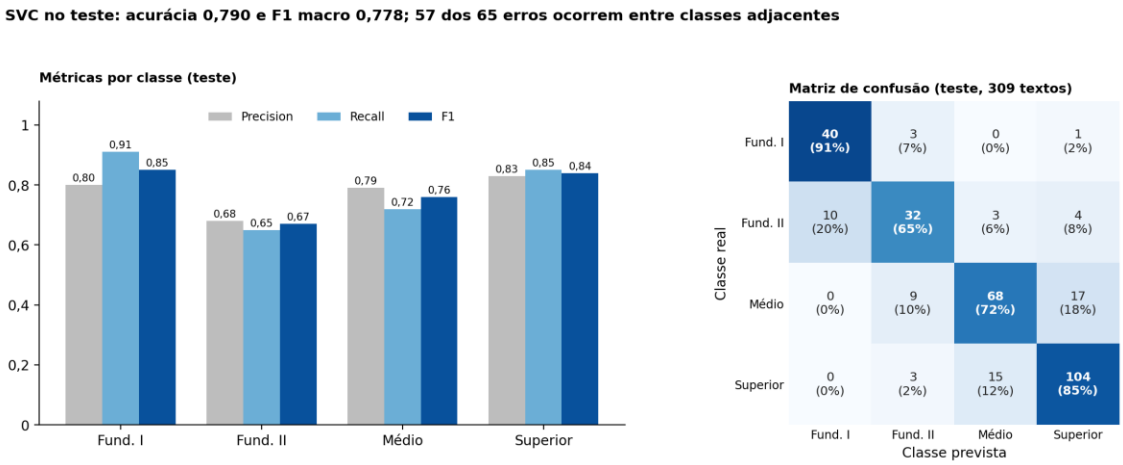



*Figura 12 — SVC no conjunto de teste (309 textos). Esquerda: precision, recall e F1 por classe. Direita: matriz de confusão (contagem e porcentagem da linha; substitui a tabela de confusão). Dos 65 erros, 57 (87,7%) ocorrem entre classes adjacentes e 8 entre classes não adjacentes; essa contagem foi derivada da matriz.*

Acertos: 40 + 32 + 68 + 104 = 244 de 309 (79,0%).

### 5.4 Interpretação

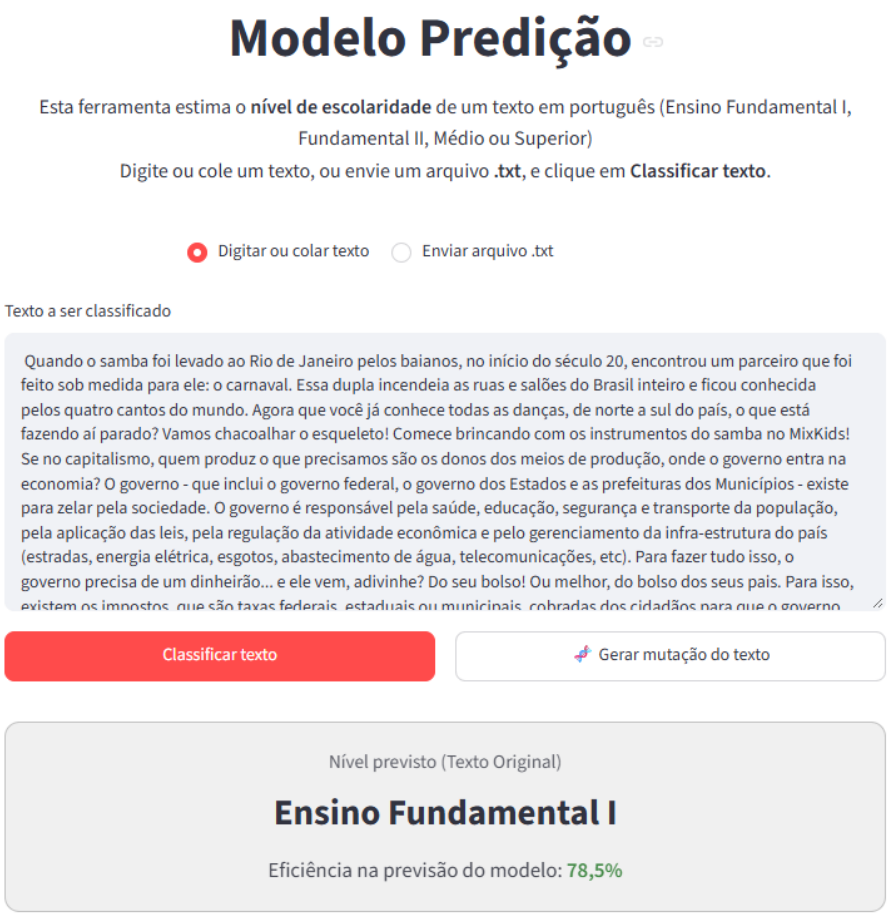
- **Erros em classes vizinhas.** Médio e Superior nunca foram classificados como Fund. I (0 e 0), e Fund. I foi classificado como Médio em 0 casos e como Superior em 1. O modelo aprendeu uma ordem de complexidade e erra, em geral, por um degrau.
- **A maior confusão continua sendo Médio ↔ Superior:** 17 + 15 = 32 erros, o mesmo padrão observado desde o baseline.
- **O gargalo por classe mudou para Fund. II** (recall 0,65). Dos 49 textos, 10 foram classificados como Fund. I. Com o Random Forest, o erro de Fund. II ia principalmente para as classes acima.
- **O Médio teve recall de 0,72**, contra 0,60 do Random Forest com 30 features. A comparação é apenas indicativa, porque usa conjuntos diferentes (validação e teste). Uma hipótese não testada é que a fronteira não linear e suave do SVC lida melhor com essa classe que as árvores.
- **Cautela com as classes menores.** Com 44 e 49 textos, cada erro vale cerca de 2 pontos de recall; a diferença entre 10 e 6 textos de Fund. II indo para Fund. I é ruído de amostra, e não se devem tirar conclusões fortes de um único teste.

# 6. Aplicação: interface web e algoritmo de mutação

A descrição desta seção baseia-se na leitura do código de app.py, Features.py e mudança_texto.py. O algoritmo de mutação **não foi avaliado quantitativamente** neste relatório (Seção 6.3).

## 6.1 Interface (Streamlit)

A interface (app.py) usa o layout centralizado do Streamlit, com título, descrição de uso, campo de entrada, botões e área de resultado centralizados. Ela importa extrair_features_nlp e NOMES_FEATURES de Features.py (o mesmo código do treino) e a função main de mudança_texto.py.




| Elemento | Comportamento |
|---|---|
| Entrada | Seletor entre texto digitado/colado e envio de arquivo .txt. A leitura do arquivo tenta utf-8-sig, cp1252 e latin-1, nessa ordem, pois arquivos em português nem sempre são UTF-8 |
| Botão **Classificar texto** | Extrai as 30 features do texto-alvo, aplica o pipeline salvo e exibe o nível previsto |

| Elemento | Comportamento |
|---|---|
| Botão **Gerar mutação do texto** | Chama o algoritmo de mutação e guarda o resultado em st.session_state['texto_modificado'] |
| Texto-alvo | O texto modificado, se existir na sessão; caso contrário, o original. O resultado informa qual foi classificado («Texto Original» ou «Texto Modificado») |
| Invalidação | Se o texto de entrada mudar em relação ao que gerou a mutação, a mutação é descartada |
| Botão **Voltar ao original** | Descarta a mutação e recarrega a página |
| Validações | Texto vazio (aviso); acima de 500.000 caracteres (erro, pois o spaCy rejeita textos com mais de 1.000.000); menos de 300 palavras (aviso de confiabilidade, mas a classificação é feita) |
| Erros | Modelo ausente (FileNotFoundError) e exceções genéricas são exibidos na interface |
| Desempenho | O modelo é carregado uma única vez (@st.cache_resource) |

O st.session_state é necessário porque o Streamlit reexecuta o script inteiro a cada clique: sem ele, o texto modificado se perderia entre o clique de mutação e o de classificação.

O fluxo de classificação repete o caminho do treino: texto → extrair_features_nlp → DataFrame com as colunas na ordem de NOMES_FEATURES → modelo.predict → rótulo (por exemplo, 3_Ensino_Medio), convertido por um dicionário em um nome legível («Ensino Médio»).

### 6.2 Algoritmo de mutação de texto

**Objetivo:** gerar uma versão modificada do texto de entrada para que o usuário possa reclassificá-la e verificar se a complexidade prevista se altera, o que serve como sonda de sensibilidade do classificador. O ponto de entrada é main(texto_usuario), que chama gerar_mutacoes_texto.

**Pipeline:**

1. **Análise.** O texto é processado pelo spaCy (pt_core_news_sm, NER desativado), que fornece classe gramatical e relações de dependência de cada token.
2. **Troca de gênero com concordância** (mutacao_troca_genero_inteligente). Para cada token que seja substantivo ou pronome e conste em MAPA_SUBSTANTIVOS (por exemplo, aluno ↔ aluna, ele ↔ ela), a forma é trocada pela do gênero oposto. A concordância é propagada pela árvore de dependências: os determinantes dependentes do núcleo (MAPA_DETERMINANTES: artigos, demonstrativos, possessivos, contrações como no/na e ao/à) e os adjetivos dependentes (MAPA_ADJETIVOS) são flexionados; se o núcleo for sujeito de um verbo ou auxiliar, os adjetivos predicativos também são flexionados.

3. **Nova análise.** O texto resultante passa novamente pelo spaCy, para que as classes gramaticais da etapa seguinte reflitam o texto já modificado.
4. **Substituição lexical** (mutacao_troca_palavras_inteligente). Cada token cuja tupla (forma em minúsculas, classe gramatical) conste em SINONIMOS_POS ou ANTONIMOS_POS torna-se candidato; com **probabilidade de 0,3** (random.random()), uma das opções compatíveis com a classe gramatical é sorteada para substituí-lo.
5. **Reconstrução.** O texto é remontado token a token com token.whitespace_, preservando espaços e pontuação, e a capitalização original (minúsculas, inicial maiúscula ou caixa alta) é mantida por manter_capitalizacao.

**Cobertura dos dicionários:**

| Dicionário | Entradas | Conteúdo |
|---|---|---|
| MAPA_SUBSTANTIVOS | 26 | 13 pares de gênero (*cantor/cantora*, *professor/professora*, *homem/mulher*, *ele/ela*, *eles/elas* etc.) |
| MAPA_DETERMINANTES | 34 | 17 pares (*o/a*, *um/uma*, *este/esta*, *meu/minha*, *pelo/pela*, *dele/dela*, *no/na*, *ao/à* etc.) |
| MAPA_ADJETIVOS | 28 | 14 pares (*famoso/famosa*, *bonito/bonita*, *novo/nova* etc.) |
| SINONIMOS_POS | 15 | Sinônimos por (palavra, classe), como (*famoso*, ADJ) e (*texto*, NOUN) |
| ANTONIMOS_POS | 9 | Antônimos por (palavra, classe), como (*grande*, ADJ) e (*novo*, ADJ) |

**Decisões de projeto:** (i) a troca de gênero usa a árvore de dependências, e não a troca isolada de palavras, para manter artigos e adjetivos concordando com o substantivo; (ii) o dicionário é indexado por (palavra, classe gramatical), o que evita substituir uma palavra em um uso gramatical errado; (iii) o texto é processado duas vezes pelo spaCy porque a segunda etapa precisa analisar o texto já alterado; (iv) a reconstrução com `whitespace_` preserva a formatação original.

**Resultado observado e conclusão:** o algoritmo cumpre a função para a qual foi codificado. Ele executa as alterações previstas (troca de gênero com propagação de concordância e substituição lexical por sinônimos e antônimos), e o texto modificado é exibido e pode ser reclassificado na interface. Porém, na avaliação qualitativa do autor, por inspeção dos textos gerados, ele **não atinge o objetivo de alterar a complexidade do texto de forma controlada**: o texto produzido não fica coeso nem coerente, e as modificações não seguem uma direção de complexidade (simplificar ou complexificar). A abordagem escolhida mostrou-se inadequada ao objetivo, em parte por ter sido

desenvolvida sob prazo apertado. Essa avaliação é qualitativa e não foi acompanhada de medição (Seção 6.3).

Contribuem para esse resultado:

- o algoritmo aplica perturbações pontuais a um texto já existente, sem nenhuma representação do conteúdo, e as substituições não consideram o contexto semântico (sinônimos e antônimos são sorteados juntos, e os antônimos invertem o sentido);
- a cobertura dos dicionários é pequena (Seção 6.2, tabela de cobertura), de modo que poucas palavras são alteradas;
- as alterações incidem pouco sobre as features usadas pelo classificador (Seção 6.3), de modo que, mesmo se fossem coerentes, tenderiam a mudar pouco a previsão.

**Melhoria proposta: geração guiada pelo conteúdo.** Em vez de aplicar mutações a um texto existente, o algoritmo passaria a gerar um texto novo a partir da análise do original, em quatro etapas:

1. identificar o **assunto central** do texto;
2. extrair os **temas mais citados** ao longo do texto;
3. levantar as **palavras mais utilizadas**, por exemplo por frequência, descontadas as palavras funcionais;
4. criar, a partir desse conteúdo, um **novo texto coeso e coerente**, em um nível de complexidade-alvo.

Nessa abordagem a complexidade passa a ser uma variável de controle (entrada do gerador), e não um efeito colateral de trocas de palavras. A avaliação também fica mais direta: comparar o nível-alvo pedido com o nível previsto pelo classificador. A implementação exigiria um componente de geração de linguagem natural, fora do escopo atual do projeto.

**Implicação para a Seção 6.3:** como a mutação atual não foi concebida para controlar a complexidade, o protocolo proposto deve ser lido como um **teste de robustez do classificador a perturbações superficiais**, e não como medida da capacidade de alterar o nível de um texto.

### 6.3 Efeito esperado sobre as features e plano de avaliação

Como o classificador recebe apenas o vetor de 30 features, a mutação só altera a previsão se alterar esse vetor. A análise abaixo é **teórica, baseada no código, e não em medições**:

| Tipo de mutação | Features potencialmente afetadas | Expectativa |
|---|---|---|
| Troca de gênero | Comprimento médio de palavra e proporção de palavras longas, apenas nos pares com comprimentos diferentes (*professor/professora*, *cantor/cantora*, *ator/atriz*, *homem/mulher*: +1 letra; *irmão/irmã*: −1) | Efeito mínimo: as classes gramaticais e a pontuação não mudam (substantivo continua substantivo, artigo continua artigo), e o tamanho das sentenças também não |
| Substituição por sinônimos/antônimos | Comprimento das palavras (por exemplo, *texto* → *documento*, de 5 para 9 letras), proporção de palavras longas, TTR e canonicidade | Efeito pequeno: só ocorre em tokens presentes nas 24 entradas dos dicionários e sorteados com probabilidade 0,3; em um texto de ~300 palavras, espera-se poucas substituições |

A hipótese resultante é que a mutação raramente altera a classe prevista, exceto em textos próximos de uma fronteira de decisão. Essa hipótese **não foi testada**. Protocolo proposto:

1. Para cada texto do conjunto de teste (309), gerar a mutação com semente fixa (random.seed), já que o algoritmo é estocástico.
2. Medir: (a) a **taxa de mudança da classe prevista**, no total e por classe; (b) o número médio de tokens alterados por texto; (c) a variação de cada feature (média e desvio de Δ), com atenção a lex_* e canonicidade; (d) a variação da função de decisão do SVC, que mostra sensibilidade mesmo sem troca de classe.
3. Repetir com várias sementes para estimar a variabilidade.
4. Separar o efeito de cada componente (apenas gênero; apenas sinônimos/antônimos) e comparar com uma mutação de controle (substituições aleatórias de palavras de mesma classe).

### 6.4 Observações técnicas sobre a interface e o algoritmo

Pontos identificados na revisão do código. Não foram executados; os itens marcados como «verificar» dependem de teste.

**Interface:**

- **A comparação original × modificado não é simultânea.** Depois da mutação, **Classificar texto** classifica apenas o texto modificado; para ver a previsão do original, o usuário precisa usar **Voltar ao original** e classificar de novo. Exibir as duas previsões lado a lado tornaria a comparação, que é o objetivo da funcionalidade, direta.

- **O campo «Texto modificado ativo» parece editável, mas não é.** Ele é um st.text_area sem key, e o valor devolvido é ignorado: edições do usuário não são gravadas, e a classificação usa o texto guardado na sessão.
- Features.py e mudança_texto.py carregam, cada um, uma instância do pt_core_news_sm, duplicando memória e tempo de carga. Além disso, os nltk.download rodam na importação e exigem rede na primeira execução.
- O nome do módulo mudança_texto tem caracteres não ASCII. Funciona no Python 3, mas pode causar problemas entre sistemas de arquivos; renomear para mudanca_texto é mais seguro. Do mesmo modo, Features.py precisa coincidir exatamente, em maiúsculas e minúsculas, com o nome usado no import, para funcionar em sistemas sensíveis à caixa.

**Algoritmo de mutação:**

- **A mutação não preserva o significado.** Os candidatos de substituição misturam sinônimos e antônimos (*famoso* pode virar *desconhecido*). Ela não é uma paráfrase nem tem direção (simplificar ou complexificar): é uma perturbação aleatória controlada.
- **Cobertura baixa.** Os dicionários têm poucas entradas e só formas exatas (por exemplo, *famosos* não tem sinônimos cadastrados), o que limita o número de substituições.
- **Não determinístico.** O sorteio não usa semente; a cada clique o resultado pode ser diferente.
- **Rótulos de dependência (verificar).** As condições acomp, attr e det:poss não pertencem, em princípio, ao esquema de dependências universais que o modelo em português usa; é provável que não disparem nunca. Isso deve ser confirmado imprimindo token.dep_ em textos de exemplo.
- **Concordância limitada ao que está nos dicionários.** Substantivos ou adjetivos fora dos mapas não são flexionados, de modo que o texto resultante pode ter inconsistências de gênero.

# 7. Discussão e limitações

### 7.1 Principais achados

- Features estruturais simples (pontuação, tamanho de sentença, classes gramaticais) separam bem o Fundamental I das demais classes (recall de 0,91 a 0,98), mas não separam Fund. II, Médio e Superior entre si.
- O TTR em janela fixa de 300 palavras foi a feature mais importante (0,101) entre as quatro features lexicais que elevaram o F1 macro em +0,031 na CV.
- O SVC com StandardScaler foi o melhor algoritmo, com desempenho em dados de teste (F1 macro 0,778) consistente com a validação cruzada.
- A estimativa de ganho dependeu do protocolo: a validação simples sugeriu +0,065, e a CV mostrou +0,031.
- Os resultados negativos (34 features, MultinomialNB, canonicidade) foram documentados com critério.

### 7.2 Canonicidade

A canonicidade foi mantida por ser requisito, mas teve baixo poder discriminativo: médias de 0,221 a 0,230 nas quatro classes e importância entre as menores no Random Forest (0,029 com 26 features; 0,019 com 30). A explicação proposta, não verificada, é que a medida por **palavra inteira** é rígida e captura sobretudo palavras funcionais curtas, comuns em qualquer nível. Uma verificação simples seria listar as palavras canônicas mais frequentes. Uma versão por **sílabas** (proporção de sílabas CV, com separação silábica real) distinguiria melhor *casa/bola* de *constituição/pragmático* e é a evolução natural.

### 7.3 Confusão entre Médio e Superior

Mesmo com o melhor modelo, 32 dos 309 textos de teste (17 + 15) foram trocados entre Médio e Superior. As features do projeto medem **forma** (estrutura da frase, classes gramaticais, comprimento e diversidade de palavras), e não **conteúdo** (tema, vocabulário técnico). Representações lexicais, como TF-IDF combinado com as 30 features, são a próxima etapa natural e o contexto em que o MultinomialNB poderia ser competitivo.

### 7.4 Risco de aprendizado por atalho

As observações da Seção 2.6 sugerem que os níveis podem vir de fontes editoriais distintas. Sinais compatíveis com isso nos resultados: pos_PROPN com pico isolado no Médio (0,079) e entre as features mais importantes do baseline; punct_aspas e punct_parenteses com saltos de 2,6 e 3,0 vezes de Fund. II para Médio. Se parte do

desempenho vier de assinaturas de fonte e de formatação, o modelo pode não generalizar para textos de outras origens, o que inclui os textos que o usuário enviar à interface. **Isto é um risco, não um resultado medido.** Verificações possíveis: remover pos_PROPN, punct_aspas e punct_parenteses e medir a queda na CV; avaliar o modelo em textos de uma fonte não vista no treino; e normalizar a formatação (remover cabeçalhos e marcadores) antes da extração.

### 7.5 Limitações de dados e de avaliação

- **Classes pequenas:** 44 e 49 textos de teste em Fund. I e II tornam as métricas por classe imprecisas (um erro ≈ 2 pontos de recall).
- **Desbalanceamento:** o MultinomialNB não pôde ser balanceado; os demais usam class_weight='balanced'.
- **Hiperparâmetros** do SVC não foram otimizados; SVC e Random Forest empatam tecnicamente na CV.
- **Divisão única de teste:** o teste foi usado uma vez, o que é correto, mas a margem de erro com 309 textos é de alguns pontos percentuais.
- **Deduplicação:** remove apenas textos idênticos; textos quase idênticos ou do mesmo documento de origem podem estar em partições diferentes, o que não foi verificado (o DataFrame guarda apenas texto e categoria, sem identificador de origem).
- **Qualidade do pré-processamento:** o modelo pequeno do spaCy (sm) comete erros de etiquetagem e segmentação, principalmente em textos com cabeçalhos e fragmentos; esses erros se propagam para as features.
- **Textos curtos e fora do domínio:** o modelo foi treinado com textos de ~300 palavras ou mais, e o TTR em 300 palavras e as proporções ficam menos confiáveis em textos curtos; a interface apenas avisa o usuário.
- **Algoritmo de mutação:** não avaliado quantitativamente (Seções 6.3 e 6.4).

# 8. Conclusão e trabalhos futuros

## 8.1 Conclusão

O projeto entregou um classificador de complexidade textual em português, uma interface web e um algoritmo de mutação integrado a ela. O modelo final (SVC sobre 30 features linguísticas) atingiu **F1 macro de 0,778 e acurácia de 0,790** em dados de teste nunca vistos, consistente com a validação cruzada (0,785 ± 0,015) e muito acima dos pisos de 25% e 39,5%. O desenvolvimento foi guiado por hipóteses testáveis: cada feature foi validada por análise descritiva e importância, cada mudança foi medida com validação cruzada pareada e uma regra de decisão baseada no desvio, e os resultados negativos (34 features, MultinomialNB, canonicidade) foram registrados.

| **Objetivo** | **Situação** |
|---|---|
| Ingestão reprodutível e divisão estratificada | Atendido (2.060 textos; 1.442 / 309 / 309) |
| Canonicidade e demais features, validadas | Atendido; a canonicidade tem baixo poder discriminativo |
| Comparação dos quatro algoritmos com a mesma CV | Atendido; SVC escolhido |
| Avaliação única no teste | Atendido (F1 macro 0,778; acurácia 0,790) |
| Interface web com entrada por texto ou .txt | Atendido; validado pelo autor em uso |
| Algoritmo de mutação e reclassificação | Implementado e integrado; **efeito não quantificado** |

As limitações principais são a confusão entre Médio e Superior (32 erros de 309), o risco de aprendizado por atalho decorrente de fontes heterogêneas, a avaliação não realizada do algoritmo de mutação e o uso de hiperparâmetros padrão.

## 8.2 Trabalhos futuros

Em ordem sugerida de prioridade:

1. **Avaliar o algoritmo de mutação** segundo o protocolo da Seção 6.3 (taxa de mudança de classe, controles e múltiplas sementes).
2. **Testar novo algoritmo de mutação** construir nova logica para modificar textos, invés de alterar estrutura, baseia-se no contexto, tema, e palavras mais utilizadas para gerar novo texto (sugestão: utilizar modelo de LLM com processo de embedding)
3. **Testar o aprendizado por atalho** (Seção 7.4): ablação de pos_PROPN, punct_aspas e punct_parenteses; avaliação em fonte não vista.
4. **Adicionar representação lexical (TF-IDF)** combinada com as 30 features, para atacar a confusão Médio ↔ Superior; reavaliar o MultinomialNB nesse contexto.

5. **Canonicidade por sílabas**, com separação silábica real, para dar à feature obrigatória um poder discriminativo maior.
6. **Ajustar hiperparâmetros** do SVC (C, gamma) com GridSearchCV e comparar SVC e Random Forest por dobra.
7. **Ablação individual** das quatro features de riqueza lexical, para separar redundância de ruído.
8. **Melhorar a interface:** exibir as previsões do original e do modificado lado a lado; tornar editável (ou explícito como somente leitura) o campo do texto modificado; permitir fixar a semente e mostrar as palavras alteradas.
9. **Explorar modelos de linguagem maiores** (modelos spaCy maiores ou representações de linguagem pré-treinadas) como referência de desempenho, ao custo de menor interpretabilidade.

## 10. Referência bibliográfica


BIRD, Steven; KLEIN, Ewan; LOPER, Edward. Natural language processing with Python. Sebastopol: O'Reilly Media, 2009.

BREIMAN, Leo. Random forests. Machine Learning, v. 45, n. 1, p. 5–32, 2001.

CÂMARA JÚNIOR, Joaquim Mattoso. Estrutura da língua portuguesa. Petrópolis: Vozes, 1970.

CORTES, Corinna; VAPNIK, Vladimir. Support-vector networks. Machine Learning, v. 20, n. 3, p. 273–297, 1995. DOI: 10.1007/BF00994018.

COVINGTON, Michael A.; MCFALL, Joe D. Cutting the Gordian knot: the moving-average type–token ratio (MATTR). Journal of Quantitative Linguistics, v. 17, n. 2, p. 94–100, 2010. DOI: 10.1080/09296171003643098.

GEIRHOS, Robert et al. Shortcut learning in deep neural networks. Nature Machine Intelligence, v. 2, n. 11, p. 665–673, 2020. DOI: 10.1038/s42256-020-00257-z.

PEDREGOSA, Fabian et al. Scikit-learn: machine learning in Python. Journal of Machine Learning Research, v. 12, p. 2825–2830, 2011.

SCARTON, Carolina Evaristo; ALUÍSIO, Sandra Maria. Análise da inteligibilidade de textos via ferramentas de processamento de língua natural: adaptando as métricas do Coh-Metrix para o português. Linguamática, v. 2, n. 1, p. 45–62, 2010. Disponível em: https://linguamatica.com/index.php/linguamatica/article/view/44. Acesso em: 16 out. 2026.

SCIKIT-LEARN. User guide. [S. l.]: scikit-learn, [s. d.]. Disponível em:

https://scikit-learn.org/stable/user_guide.html

SPACY. Portuguese. [S. l.]: spaCy, [s. d.]. Disponível em:

https://spacy.io/models/pt

STREAMLIT. Streamlit documentation. [S. l.]: Streamlit, [s. d.]. Disponível em:

https://docs.streamlit.io/

# 10. Anexos

### 10.1 Estrutura do projeto e execução

projeto/
├── construindo_ML.ipynb  # pesquisa: dados, features, experimentos, treino e teste
├── Features.py  # extração das 30 features
├── mudança_texto.py  # algoritmo de mutação de texto
├── app.py  # interface Streamlit
└── modelo_svc_complexidade.joblib  # pipeline treinado (StandardScaler + SVC)

Para executar a interface, na pasta do projeto:

pip install streamlit
streamlit run app.py

O pt_core_news_sm precisa estar instalado (python -m spacy download pt_core_news_sm), e a primeira execução baixa os recursos punkt e punkt_tab do NLTK (exige rede).

### 10.2 Ambiente

- Python 3.14.6 (conforme os metadados do notebook).
- Versões das bibliotecas principais, extraídas do requirements.txt do ambiente do projeto (tabela abaixo). O arquivo .joblib depende da versão do scikit-learn usada no treino; recomenda-se executar a interface com a mesma versão.

| Biblioteca | Versão | Uso no projeto |
|---|---|---|
| pandas | 3.0.6 | DataFrames |
| numpy | 2.5.3 | Cálculo numérico |
| scikit-learn | 1.9.1 | Modelos, validação cruzada, métricas e Pipeline |
| joblib | 1.6.0 | Persistência do modelo |
| spaCy | 3.8.16 | Processamento de linguagem natural |
| pt_core_news_sm | 3.8.0 | Modelo do spaCy para português |
| NLTK | 3.10.3 | Tokenização na canonicidade |
| Streamlit | 1.66.0 | Interface web |
| SciPy | 1.18.1 | Dependência do scikit-learn |

### 10.3 Configuração do modelo final

```
modelo_final = make_pipeline(
  StandardScaler(),
  SVC(class_weight="balanced", random_state=42),
)
modelo_final.fit(X_final, y_final)   # treino + validação (1.751 textos)
```

### 10.4 As 30 features, em ordem

punct_virgula, punct_ponto_final, punct_reticencias, punct_interrogacao,
punct_exclamacao,
punct_ponto_virgula, punct_dois_pontos, punct_aspas, punct_parenteses,
punct_travessao,
sent_tam_medio, sent_tam_desvio,
pos_NOUN, pos_VERB, pos_ADJ, pos_ADV, pos_PRON, pos_DET, pos_ADP, pos_CCONJ,
pos_SCONJ,
pos_AUX, pos_NUM, pos_PROPN, pos_INTJ,
lex_tam_medio_palavra, lex_desvio_tam_palavra, lex_prop_palavras_longas, lex_ttr_300,
canonicidade

### 10.5 Tecnologias utilizadas

Python; pandas e numpy (dados e cálculo); scikit-learn (modelos, validação cruzada, métricas, Pipeline); spaCy com pt_core_news_sm (tokenização, classes gramaticais, segmentação de sentenças e dependências); NLTK (tokenização na canonicidade); joblib (persistência do modelo); Streamlit (interface web).